

HappyHeadlines — Cache Layers

DLS Compulsory Assignment #1 (W40) · Presentation & finished solution · Repo:
github.com/BenjaminDamC/happy-headlines-cache

The 10-minute pitch in one breath: HappyHeadlines' services are already replicated to every region, but they all read from a single database in North America, so European users get slow responses and occasional outages. Replicating the database was rejected on cost. So I put a **cache layer in front of each database**: an **ArticleCache** (offline-filled with the latest 14 days of articles) and a **CommentCache** (cache-miss with LRU over the 30 most recently accessed articles). Both are implemented, containerized with Docker Compose, and verified.

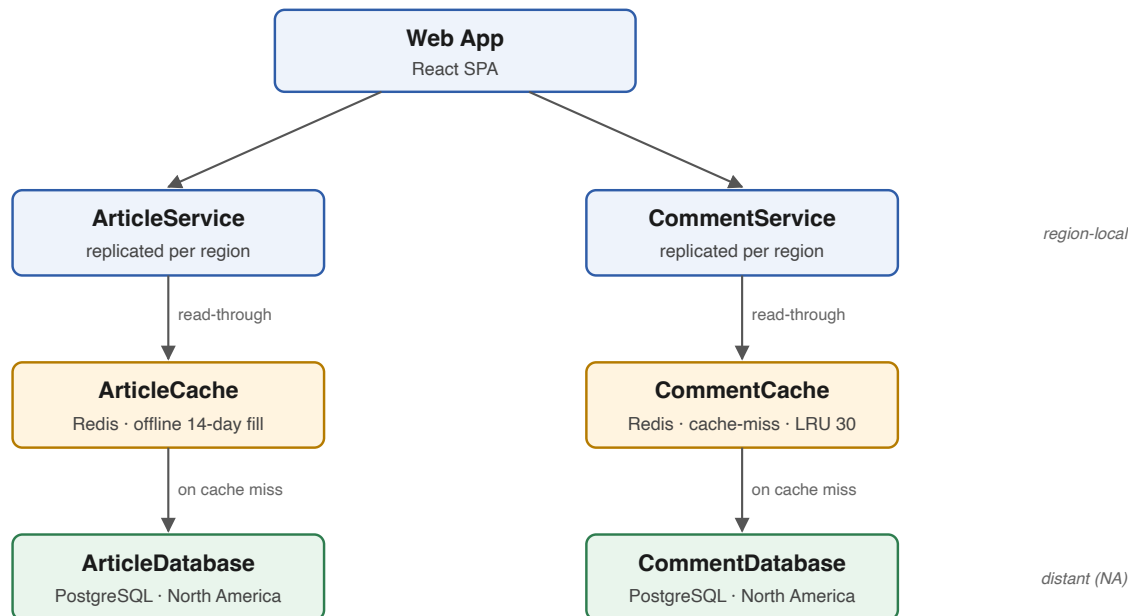
1 · The assignment

- Fix HappyHeadlines' availability + latency by adding two cache layers.
- **ArticleCache** — between ArticleService and ArticleDatabase; an *offline* process periodically fills it with articles from the latest 14 days.
- **CommentCache** — between CommentService and CommentDatabase; *cache-miss* fill, limited to the 30 most recently accessed articles, LRU eviction.
- A small **dashboard** showing each cache's hit ratio (the remaining piece).
- Present on Zoom, ≤10 min, screen sharing, webcam on, individual pass/fail.

2 · The problem

The ArticleService and CommentService are already replicated to each region, but they all talk to a single global database instance in North America. European users see slow responses; some articles become unavailable. Replicating the database (an x-axis split) was rejected on cost, so the ARB approved **cache layers** in front of each database instead.

3 · Architecture



4 · The two cache strategies

ArticleCache — offline fill

A background process (`article-fill`) periodically copies the latest 14 days of articles into Redis. The service only *reads* the cache and falls back to the database for older articles. Readers get a fast, region-local read for recent content.

CommentCache — cache-miss + LRU

On a miss the service reads the database *and* stores the comments, tracking the most-recently-accessed articles in a sorted set. When more than 30 articles are cached, the least-recently-used is evicted.

5 · Rehearsal script (10 min)

- **0:00–0:30 — Intro.** "Hi, I'm Benjamin. I'll show how I fixed HappyHeadlines' availability with two cache layers."
- **0:30–2:30 — The problem.** Services are replicated per region, but both talk to one global database in North America → high latency + outages for European users. The ARB rejected an x-axis split on cost. (Point at the region-vs-North-America split on the diagram.)
- **2:30–6:00 — The solution.** Two cache layers: **ArticleCache** (offline process fills the last 14 days) and **CommentCache** (cache-miss; holds the 30 most recent articles; evicts LRU when full). (Walk the container diagram left-to-right: service → cache → DB.)
- **6:00–8:30 — Live demo.** `docker compose up`; show `/articles/1` → `source:"cache"`, `/articles/12` → `source:"db"`, and the comment cache-miss → hit on the second request.
- **8:30–10:00 — Wrap-up.** "Both cache layers are implemented and verified. The one piece still left is the hit-ratio dashboard."

6 · Likely questions (+ answers)

- **Why not just replicate the database?** Cost — the ARB rejected an x-axis split because of the expense of standing up and maintaining DBs in Europe and elsewhere.
- **Why a cache instead of a read replica?** A cache is cheaper and sits where the readers are; it trades a little freshness (stale-by-design) for low latency and high availability.

- **Difference between the two caches?** ArticleCache is *offline filled* (a background process pre-warms the last 14 days). CommentCache is *cache-miss* (filled on demand, capped at 30 articles, LRU eviction).
- **What does LRU mean here?** When the comment cache is full, the article that hasn't been accessed for the longest is dropped to make room.
- **Why Redis?** The brief allows any tool; Redis is the common choice for an in-memory cache with built-in expiry and LRU eviction.
- **How would you measure success?** The hit-ratio dashboard — the fraction of requests served from cache vs. the database (the remaining piece).
- **What about stale data?** The 14-day article cache is stale-by-design; for comments, the LRU cap bounds staleness. Freshness vs. speed is the trade-off.

7 • Presentation rules

- Join only in your assigned timeslot (not early; leave on time).
- Webcam on for the whole meeting.
- Start by saying your name + a short intro.
- Screen sharing active, ≤ 10 minutes.
- Ask a curiosity question to at least one other presenter after.

8 · The solution — full source

Docker Compose topology docker-compose.yml

```
# HappyHeadlines – cache layers between the region-replicated services and the
# distant global databases. DLS Compulsory Assignment #1 (W40).

services:
  # --- Global databases (in North America; NOT replicated to regions) ---
  article-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: articles
      POSTGRES_USER: hh
      POSTGRES_PASSWORD: hh
    ports: ["5433:5432"]
    volumes:
      - ./db/init-article.sql:/docker-entrypoint-initdb.d/init.sql:ro

  comment-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: comments
      POSTGRES_USER: hh
      POSTGRES_PASSWORD: hh
    ports: ["5434:5432"]
    volumes:
      - ./db/init-comment.sql:/docker-entrypoint-initdb.d/init.sql:ro

  # --- Cache layers ---
  article-cache:
    image: redis:7-alpine
    ports: ["6379:6379"]

  comment-cache:
    image: redis:7-alpine
    ports: ["6380:6379"]

  # --- Services (replicated per region; talk to the distant DBs via the caches) ---
  article-service:
    build: ./server/article-service
    ports: ["4001:4001"]
    depends_on: [article-cache, article-db]
    environment:
      REDIS_URL: redis://article-cache:6379
      DATABASE_URL: postgres://hh:hh@article-db:5432/articles

  # The ArticleCache OFFLINE fill process – periodically copies the latest
  # 14 days of articles into Redis. Runs independently of the service.
  article-fill:
    build: ./server/article-service
    command: ["node", "fill-cache.js"]
    depends_on: [article-cache, article-db]
    environment:
      REDIS_URL: redis://article-cache:6379
      DATABASE_URL: postgres://hh:hh@article-db:5432/articles
      FILL_INTERVAL_MS: 30000

  comment-service:
    build: ./server/comment-service
    ports: ["4002:4002"]
    depends_on: [comment-cache, comment-db]
    environment:
      REDIS_URL: redis://comment-cache:6379
      DATABASE_URL: postgres://hh:hh@comment-db:5432/comments
```

C4 model (Structurizr DSL) docs/architecture.dsl

```
workspace "HappyHeadlines – Cache Layers" "DLS Compulsory Assignment #1 (W40): availability fix via cache layers between the
region-replicated services and the North-America-only global databases." {

  model {
    rider = person "Rider" "Reads news and comments"
    editor = person "Editor" "Publishes articles"

    hh = softwareSystem "HappyHeadlines" "News platform" {
      web = container "Web App" "React single-page app"

      articleSvc = container "ArticleService" "Serves articles. Already replicated to each region." "Node.js"
```

```

        commentSvc = container "CommentService" "Serves comments. Already replicated to each region." "Node.js"

        articleCache = container "ArticleCache" "Redis. Offline process fills it with articles from the latest 14 days."
"Redis"
        commentCache = container "CommentCache" "Redis. Cache-miss fill; holds comments for the 30 most recently accessed
articles; LRU eviction." "Redis"

        articleDb = container "ArticleDatabase" "PostgreSQL. Single global instance in North America." "PostgreSQL"
        commentDb = container "CommentDatabase" "PostgreSQL. Single global instance in North America." "PostgreSQL"
    }

    rider -> web          "reads news"
    editor -> web          "publishes articles"
    web    -> articleSvc "HTTPS"
    web    -> commentSvc "HTTPS"

    # The cache layers sit between the region-local services and the
    # distant North-America databases – this is the whole fix.
    articleSvc -> articleCache "read-through cache"
    articleSvc -> articleDb    "on cache miss"
    commentSvc -> commentCache "read-through cache"
    commentSvc -> commentDb    "on cache miss"
}

views {
    systemContext hh "System Context" {
        include *
    }
    container hh "Containers (cache layers highlighted)" {
        include *
    }
}
}

```

README README.md

HappyHeadlines – Cache Layers

DLS Compulsory Assignment #1 (W40): fix HappyHeadlines' availability and latency by putting cache layers between the region-replicated services and the distant global databases.

Both cache layers are implemented and runnable (see [Running it](#running-it)). The only piece still to add is the cache hit-ratio dashboard.

The problem

The `ArticleService` and `CommentService` are already replicated to each region, but they all talk to a single global database instance sitting in North America. European users see slow responses; some articles become unavailable. Replicating the database (an x-axis split) was rejected on cost, so the ARB approved **cache layers** in front of each database instead.

Architecture (C4)

```

```mermaid
flowchart LR
 Web["Web App"] --> AS["ArticleService
(replicated per region)"]
 Web --> CS["CommentService
(replicated per region)"]

 AS --> AC["ArticleCache
Redis · offline 14-day fill"]
 AS --> AD["ArticleDatabase
PostgreSQL · North America"]

 CS --> CC["CommentCache
Redis · cache-miss · LRU 30"]
 CS --> CD["CommentDatabase
PostgreSQL · North America"]
```

```

- **ArticleCache** – an **offline** process (`article-fill`) periodically copies the latest 14 days of articles into Redis. The service only reads the cache and falls back to the database for older articles.
- **CommentCache** – a **cache-miss** strategy: on a miss the service reads the database and stores the comments, tracking the most-recently-accessed articles in a sorted set. When more than 30 articles are cached, the least-recently-used is evicted.

The full text-based C4 model (context + container levels) is in `[docs/architecture.dsl](docs/architecture.dsl)` – open it with Structurizr Lite (`structurizr/lite` on port 8080) to browse it interactively.

Topology (Docker Compose)

| Container | Image | Role |
|----------------------------------|----------------------|---|
| `article-db`, `comment-db` | `postgres:16-alpine` | global databases (North America), seeded via <code>db/init-*.sql</code> |
| `article-cache`, `comment-cache` | `redis:7-alpine` | the two cache layers |

```
| `article-service`, `comment-service` | custom Node | region services (read-through / cache-miss) |
| `article-fill` | custom Node | the offline 14-day fill process |
```

Running it

```
```bash
docker compose up --build
```
```

Then exercise the two strategies:

```
```bash
ArticleCache (offline fill)
curl localhost:4001/cache/size # how many articles the fill cached
curl localhost:4001/articles/1 # recent -> "source":"cache"
curl localhost:4001/articles/12 # old -> "source":"db" (outside 14 days)

CommentCache (cache-miss + LRU)
curl localhost:4002/articles/1/comments # first request -> "source":"db"
curl localhost:4002/articles/1/comments # second request -> "source":"cache"
curl localhost:4002/cache/stats # cached articles vs the 30-article cap
```
```

Scope

Done: C4 model, Docker Compose topology, and both cache layers
(offline 14-day fill + cache-miss LRU).

Deferred: the cache hit-ratio dashboard (the monitoring piece of the assignment).

Article database seed [db/init-article.sql](#)

```
-- ArticleDatabase seed: articles with published_at spread over the last ~3 weeks.
-- Articles older than 14 days are intentionally included so the offline
-- ArticleCache fill (latest 14 days) visibly does NOT cache them.
CREATE TABLE articles (
  id          SERIAL PRIMARY KEY,
  title       TEXT NOT NULL,
  content     TEXT NOT NULL,
  published_at TIMESTAMPTZ NOT NULL
);

INSERT INTO articles (title, content, published_at) VALUES
('City opens new tram line', 'The new north-south tram line opened today...', NOW() - interval '1 day'),
('Airport strike averted', 'Unions reached a deal hours before the deadline...', NOW() - interval '2 days'),
('Local team wins cup final', 'A last-minute goal sealed the victory...', NOW() - interval '3 days'),
('Housing prices keep climbing', 'Prices rose for the ninth month in a row...', NOW() - interval '4 days'),
('New library opens downtown', 'The renovated main library reopened with a digital lab...', NOW() - interval '5 days'),
('Flood warning issued', 'Heavy rain expected over the weekend...', NOW() - interval '7 days'),
('Tech hub announces expansion', 'The company will hire 200 engineers...', NOW() - interval '9 days'),
('Marathon draws record crowd', 'Over 30,000 runners took part...', NOW() - interval '11 days'),
('Energy prices fall', 'Wholesale prices dropped sharply this week...', NOW() - interval '13 days'),
('Museum night a hit', 'Tens of thousands visited museums after dark...', NOW() - interval '14 days'),
('Election campaign kicks off', 'Parties launch their platforms ahead of the vote...', NOW() - interval '18 days'),
('Old bridge to be replaced', 'The century-old bridge is slated for demolition...', NOW() - interval '25 days');
```

Comment database seed [db/init-comment.sql](#)

```
-- CommentDatabase seed: comments attached to articles (referenced by article id,
-- which matches the ArticleDatabase ids by convention in this demo).
CREATE TABLE comments (
  id          SERIAL PRIMARY KEY,
  article_id  INT NOT NULL,
  author     TEXT NOT NULL,
  content    TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO comments (article_id, author, content) VALUES
(1, 'Alice', 'Finally! Been waiting for this line for years.'),
(1, 'Bob', 'Great, but will it run on time?'),
(2, 'Carla', 'Relieved. I fly out this weekend.'),
(3, 'Dan', 'What a match. Still can not believe it.'),
(3, 'Eva', 'Deserved win after that season.'),
(4, 'Frank', 'Rents are getting out of hand.'),
(5, 'Grace', 'The digital lab is a great addition.'),
(6, 'Heidi', 'Stay safe everyone.'),
(7, 'Ivan', 'More jobs is always good news.'),
(8, 'Julia', 'The atmosphere was amazing.'),
(9, 'Karl', 'Finally some relief on the bill.'),
(10, 'Lena', 'Already looking forward to next year.');
```

ArticleService — Dockerfile server/article-service/Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package.json ./
RUN npm install
COPY server.js fill-cache.js ./
EXPOSE 4001
CMD ["node", "server.js"]
```

ArticleService — package.json server/article-service/package.json

```
{
  "name": "article-service",
  "version": "1.0.0",
  "type": "module",
  "dependencies": {
    "redis": "^4.6.13",
    "pg": "^8.11.3"
  }
}
```

ArticleService — read-through server/article-service/server.js

```
// ArticleService — reads articles through the ArticleCache (Redis).
// The cache is filled OFFLINE (see fill-cache.js), never on-demand: this service
// only reads Redis and falls back to the database on a miss (e.g. an article
// outside the 14-day window).
import http from 'node:http';
import { createClient } from 'redis';
import pg from 'pg';

const { Pool } = pg;
const REDIS_URL = process.env.REDIS_URL || 'redis://localhost:6379';
const DATABASE_URL = process.env.DATABASE_URL || 'postgres://hh:hh@localhost:5432/articles';

const redis = createClient({ url: REDIS_URL });
redis.on('error', (e) => console.error('[redis]', e.message));
const pool = new Pool({ connectionString: DATABASE_URL });

async function retry(fn, label, tries = 40) {
  for (let i = 0; i < tries; i++) {
    try { await fn(); console.log(`${label} connected`); return; }
    catch { await new Promise(r => setTimeout(r, 1000)); }
  }
  throw new Error(`${label} never connected`);
}

const json = (res, code, obj) => {
  res.writeHead(code, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(obj));
};

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://x');

  if (url.pathname === '/health') {
    return json(res, 200, { status: 'ok', service: 'article-service', cache: REDIS_URL });
  }

  if (url.pathname === '/cache/size') {
    const keys = await redis.keys('article:*');
    return json(res, 200, { cachedArticles: keys.length });
  }

  if (url.pathname === '/articles') {
    const { rows } = await pool.query('SELECT id, title, published_at FROM articles ORDER BY published_at DESC');
    const cached = new Set((await redis.keys('article:*')).map(k => k.split(':')[1]));
    const articles = rows.map(a => ({ ...a, cached: cached.has(String(a.id)) }));
    return json(res, 200, { articles });
  }

  const m = url.pathname.match(/^\/articles\/(\d+)\$/);
  if (m) {
    const id = m[1];
    const cached = await redis.get(`article:${id}`);
    if (cached) return json(res, 200, { source: 'cache', article: JSON.parse(cached) });

    const { rows } = await pool.query('SELECT id, title, content, published_at FROM articles WHERE id = $1', [id]);
    if (!rows[0]) return json(res, 404, { error: 'not found' });
    return json(res, 200, { source: 'db', article: rows[0] });
  }
});
```

```

    json(res, 404, { error: 'not found' });
  });

  async function main() {
    await retry(() => redis.connect(), 'redis');
    await retry(() => pool.query('SELECT 1'), 'postgres');
    server.listen(4001, () => console.log('ArticleService listening on 4001'));
  }
  main();

```

ArticleCache — offline fill server/article-service/fill-cache.js

```

// ArticleCache OFFLINE fill – the background process that periodically copies the
// latest 14 days of articles from the database into Redis. This is the key
// difference from the CommentCache: the cache is pre-populated here, so the
// service never writes to it (a read of a non-cached article falls through to DB).
import { createClient } from 'redis';
import pg from 'pg';

const { Pool } = pg;
const REDIS_URL = process.env.REDIS_URL || 'redis://localhost:6379';
const DATABASE_URL = process.env.DATABASE_URL || 'postgres://hh:hh@localhost:5432/articles';
const INTERVAL_MS = parseInt(process.env.FILL_INTERVAL_MS || '30000', 10);

const redis = createClient({ url: REDIS_URL });
redis.on('error', (e) => console.error('[redis]', e.message));
const pool = new Pool({ connectionString: DATABASE_URL });

async function retry(fn, label, tries = 40) {
  for (let i = 0; i < tries; i++) {
    try { await fn(); console.log(`${label} connected`); return; }
    catch { await new Promise(r => setTimeout(r, 1000)); }
  }
  throw new Error(`${label} never connected`);
}

async function fill() {
  const { rows } = await pool.query(
    `SELECT id, title, content, published_at FROM articles
     WHERE published_at >= NOW() - interval '14 days'
     ORDER BY published_at DESC`
  );
  for (const a of rows) {
    await redis.set(`article:${a.id}`, JSON.stringify(a));
  }
  console.log(`[fill] cached ${rows.length} articles from the last 14 days`);
  return rows.length;
}

async function main() {
  await retry(() => redis.connect(), 'redis');
  await retry(() => pool.query('SELECT 1'), 'postgres');
  await fill();
  setInterval(fill, INTERVAL_MS);
}
main();

```

CommentService — Dockerfile server/comment-service/Dockerfile

```

FROM node:22-alpine
WORKDIR /app
COPY package.json ./
RUN npm install
COPY server.js ./
EXPOSE 4002
CMD ["node", "server.js"]

```

CommentService — package.json server/comment-service/package.json

```

{
  "name": "comment-service",
  "version": "1.0.0",
  "type": "module",
  "dependencies": {
    "redis": "^4.6.13",
    "pg": "^8.11.3"
  }
}

```

```
// CommentService - cache-miss with LRU eviction.
// Reads comments through the CommentCache (Redis). On a MISS it fetches from the
// database AND stores the result, tracking the most-recently-accessed articles in
// a sorted set. When more than 30 articles are cached, the least-recently-used one
// is evicted.
import http from 'node:http';
import { createClient } from 'redis';
import pg from 'pg';

const { Pool } = pg;
const MAX_CACHED_ARTICLES = 30;
const LRU_KEY = 'commentcache:lru';
const REDIS_URL = process.env.REDIS_URL || 'redis://localhost:6379';
const DATABASE_URL = process.env.DATABASE_URL || 'postgres://hh:hh@localhost:5432/comments';

const redis = createClient({ url: REDIS_URL });
redis.on('error', (e) => console.error('[redis]', e.message));
const pool = new Pool({ connectionString: DATABASE_URL });

async function retry(fn, label, tries = 40) {
  for (let i = 0; i < tries; i++) {
    try { await fn(); console.log(`${label} connected`); return; }
    catch { await new Promise(r => setTimeout(r, 1000)); }
  }
  throw new Error(`${label} never connected`);
}

const json = (res, code, obj) => {
  res.writeHead(code, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(obj));
};

async function getCommentsFromDb(articleId) {
  const { rows } = await pool.query(
    `SELECT id, article_id, author, content, created_at FROM comments WHERE article_id = $1 ORDER BY created_at`,
    [articleId]
  );
  return rows;
}

async function evictIfOver() {
  const count = await redis.zCard(LRU_KEY);
  if (count > MAX_CACHED_ARTICLES) {
    const oldest = await redis.zRange(LRU_KEY, 0, 0);
    if (oldest.length) {
      await redis.del(`comments:${oldest[0]}`);
      await redis.zRem(LRU_KEY, oldest[0]);
      console.log(`[lru] evicted article ${oldest[0]} (${count - 1}/${MAX_CACHED_ARTICLES} kept)`);
    }
  }
}

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://x');

  if (url.pathname === '/health') {
    return json(res, 200, { status: 'ok', service: 'comment-service', cache: REDIS_URL });
  }

  if (url.pathname === '/cache/stats') {
    const cachedArticles = await redis.zCard(LRU_KEY);
    return json(res, 200, { cachedArticles, max: MAX_CACHED_ARTICLES });
  }

  const m = url.pathname.match(/^\/articles\/(\d+)\\/comments$/);
  if (m) {
    const articleId = m[1];
    const cached = await redis.get(`comments:${articleId}`);
    if (cached) {
      await redis.zAdd(LRU_KEY, { score: Date.now(), value: articleId }); // touch → becomes most-recent
      return json(res, 200, { source: 'cache', comments: JSON.parse(cached) });
    }

    const comments = await getCommentsFromDb(articleId);
    await redis.set(`comments:${articleId}`, JSON.stringify(comments));
    await redis.zAdd(LRU_KEY, { score: Date.now(), value: articleId });
    await evictIfOver();
    return json(res, 200, { source: 'db', comments });
  }

  json(res, 404, { error: 'not found' });
});

async function main() {
```

```
    await retry(() => redis.connect(), 'redis');
    await retry(() => pool.query('SELECT 1'), 'postgres');
    server.listen(4002, () => console.log('CommentService listening on 4002'));
  }
  main();
```